Understanding GPU Architecture for AI

From Warps and Tensor Cores to Memory Hierarchy and Throughput

---

A technical handbook on how AI workloads map to real GPU hardware: streaming multiprocessors, warps, schedulers, execution pipelines, Tensor Cores, registers, shared memory, caches, HBM, occupancy, latency hiding, and performance metrics.

@techNmak

How to read this handbook

GPU architecture is easiest to reason about when four different questions are kept separate:

1. **What work exists?** The program and kernel define threads, blocks, instructions, dependencies, and memory accesses.
2. **What work is resident?** Registers, shared memory, architectural limits, and launch configuration determine how many blocks and warps can live on an execution engine at once.
3. **What work is ready to issue?** A resident warp can still be stalled on data, dependencies, synchronization, or another constraint. Schedulers choose among eligible work.[1,3]
4. **Which hardware resource is limiting progress?** Arithmetic pipelines, Tensor Cores, memory bandwidth, latency, instruction issue, synchronization, or host-side overhead can become the bottleneck under different workloads.[2,3]

The central lesson is that a GPU is a throughput machine, not a single giant arithmetic unit. A high peak FLOP rate, high occupancy, or a dashboard showing "100% GPU utilization" does not by itself establish that an AI workload is running efficiently. Those quantities measure different things.[2,4]

> **Scope.** This handbook establishes the execution mental model needed by the rest of the Understanding AI Systems series. Memory hierarchy, Roofline analysis, coalescing, shared-memory tiling, register/occupancy tuning, Tensor Core programming, and profiling each receive deeper standalone handbooks later. Here they are introduced only far enough to make GPU architecture coherent.

Vendor convention. NVIDIA CUDA terminology is used for the concrete execution path because it is widely encountered in AI systems. Vendor-specific facts are labeled. AMD terminology is used as a cross-check so that general GPU principles are not accidentally presented as CUDA laws.[8,9]

Notation. On NVIDIA hardware, a warp contains 32 threads. For a block with T threads,

$$N_{\text{warps}} = \left\lceil \frac{T}{32} \right\rceil .$$

The exact resource limits of an SM, the number and type of execution pipelines, cache organization, and matrix instructions vary by architecture generation.[1]

Contents

1	What GPU architecture actually means for AI	5
2	CPU and GPU optimize different objectives	6
3	From a kernel launch to hardware work	7
4	The Streaming Multiprocessor is the execution home	8
5	Warps and SIMT	9
6	Divergence and active lanes	10
7	Warp schedulers and eligible warps	11
8	Latency hiding is not latency reduction	12
9	Residency, execution, and progress	13
10	Occupancy: the exact definition	14
11	Why 100% occupancy is not the goal	15
12	Registers: fast, thread-local state	16
13	Register pressure, spilling, and local memory	17
14	Shared memory: software-managed on-chip storage	18
15	Caches: L1 and L2 are not shared memory	19
16	HBM and device memory: capacity is not bandwidth	20
17	The memory hierarchy is a data-movement problem	21
18	Coalescing: how a warp asks for memory	22
19	Execution pipelines: instructions do not use one resource	23
20	Tensor Cores: matrix MMA, not magic AI cores	24
21	Precision changes the available execution path	25
22	How GEMM maps onto the GPU hierarchy	26
23	Instruction-level and thread-level parallelism	27
24	Synchronization can expose idle time	28

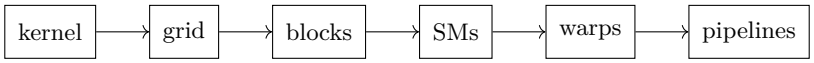
Contents continued

25	Kernel launches and asynchronous execution	29
26	What “GPU utilization” actually means	30
27	Occupancy, activity, and throughput are different	31
28	Peak TFLOPs vs achieved throughput	32
29	Compute-bound and memory-bound are workload properties	33
30	AI workloads stress different parts of the GPU	34
31	NVIDIA vs AMD: names differ, principles persist	35
32	Architecture generations change the mechanisms	36
33	Worked execution example: 61,440 logical threads	37
34	Worked occupancy example: resource limits interact	38
35	Common misconceptions	39
36	A practical mental model	40
	References and further reading	41
	Reference map: which source answers which question	42

The first half of the handbook explains the machine. The second half explains why common performance numbers can disagree. That separation is deliberate: architecture describes what resources exist; performance analysis asks which of those resources actually limited a particular workload.

1 What GPU architecture actually means for AI

An AI model is expressed as tensor operations, but hardware does not execute “a Transformer layer” or “an attention block” as a primitive. Frameworks, compilers, and libraries ultimately turn work into device operations and kernels. Those kernels expose parallel threads, instructions, memory accesses, and synchronization to a GPU execution system.[1]



The abstraction is hierarchical because the hardware is hierarchical. Blocks provide a unit that can be placed on an SM. Warps provide the native thread grouping used by the SIMT execution model. Registers and shared memory provide on-chip state. Caches and device memory feed data. Different pipelines execute different instruction classes.[1]

That is why two kernels with the same FLOP count can have very different runtimes. They may generate different memory traffic, expose different amounts of independent work, use different execution units, or suffer different amounts of synchronization and divergence. The GPU’s peak capabilities describe ceilings. The workload’s mapping onto the architecture determines how much useful work approaches those ceilings.[2,3]

Mental model. GPU architecture is the machinery that turns parallel work plus data movement into throughput. AI performance is a property of the workload-hardware mapping, not of a single specification number.

2 CPU and GPU optimize different objectives

CPU and GPU designs emphasize different kinds of progress. A CPU invests heavily in making a relatively small set of instruction streams advance quickly. A GPU accepts lower per-thread performance so it can keep a much larger population of threads in flight and deliver high aggregate throughput.[1,2]

CPU emphasis: low latency for a small number of threads

GPU emphasis: high aggregate throughput from many concurrent threads

Neither objective is universally better. Serial control-heavy code can be a poor GPU workload. Highly parallel matrix and tensor operations can be excellent GPU workloads because they provide many independent operations and regular data structures that can be mapped onto wide execution resources.

AI workloads also contain both kinds of behavior. Dense matrix multiplication exposes enormous regular parallelism. Reductions require coordination. Embedding lookups may involve irregular memory access. Autoregressive serving can have small per-request work at each decode step. The same GPU can therefore be spectacularly efficient for one kernel and underutilized for the next.

"GPU = parallel, CPU = serial" is too crude. Modern CPUs are parallel, and GPUs execute control flow. The important distinction is the amount, granularity, and scheduling style of parallel work each architecture is built to sustain.

3 From a kernel launch to hardware work

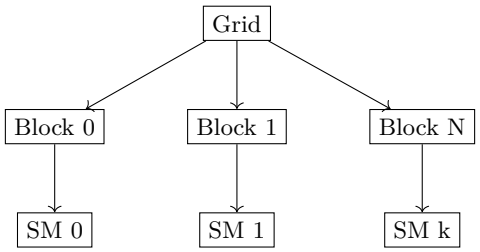
A CUDA kernel launch creates a grid of thread blocks. Each block contains threads, and blocks are scheduled onto Streaming Multiprocessors (SMs). Once resident, the threads of a block are partitioned into 32-thread warps on NVIDIA hardware.[1]

For a block containing T threads,

$$N_{\text{warps}} = \left\lceil \frac{T}{32} \right\rceil .$$

A 256-thread block contains eight warps. A 300-thread block occupies ten warps, with only 12 valid threads in the final warp. CUDA permits such a block, but the unused lanes make the final warp less efficient.[1]

A grid may contain far more blocks than can be resident simultaneously. That is normal. The grid describes the logical work. The GPU admits blocks as SM resources become available, executes them, and admits more blocks later. An ordinary block is placed on one SM for its lifetime; multiple blocks may be resident on one SM if resource limits allow.[1,2]

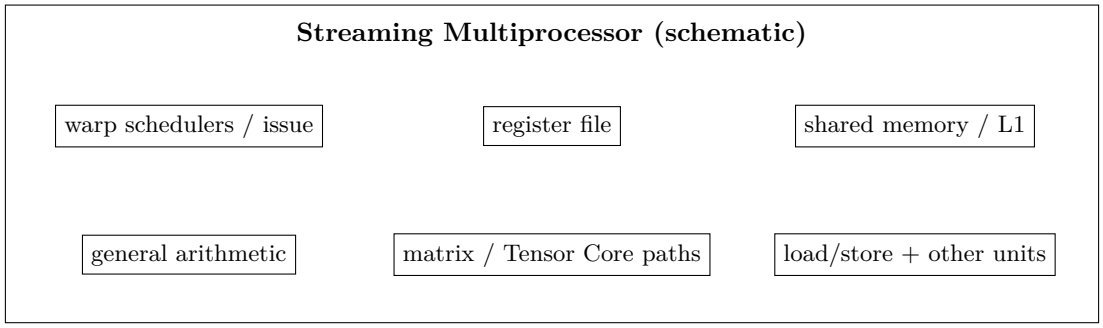


The programmer specifies the grid and block shape. The hardware/runtime determines where blocks run subject to architectural and resource constraints. This separation is one of CUDA's scalability ideas: a kernel can describe much more parallel work than the current device can execute at once.[1]

4 The Streaming Multiprocessor is the execution home

The Streaming Multiprocessor is the central execution engine in NVIDIA's architecture. It is not just a bundle of arithmetic units. It contains the resources needed to keep many warps resident and make progress: a register file, shared-memory/L1 resources, warp scheduling and issue machinery, load/store paths, and multiple classes of functional units. Exact organization varies by compute capability.[1]

A useful abstract SM looks like this:



A kernel's blocks consume finite SM resources. Registers are allocated for threads. Shared memory is allocated for blocks. Architectural limits cap resident blocks, warps, and threads. Those constraints jointly determine how much work can be resident.[1,2]

The SM model also explains why "CUDA core count" is an incomplete performance model. A workload does not consume one generic pool of identical cores. Different instruction types exercise different pipelines, and modern AI kernels can depend heavily on matrix instructions, load/store throughput, shared-memory behavior, synchronization, and scheduling. The useful unit of reasoning is the whole SM plus its memory feeds, not a single marketing count.

5 Warps and SIMT

Within an NVIDIA thread block, threads are organized into groups of 32 called warps. CUDA describes execution using SIMT: Single Instruction, Multiple Threads. Every thread runs the same kernel program, but each thread has its own data and can follow a different control-flow path.[1]

SIMT resembles SIMD because many lanes can execute a common instruction together, but CUDA explicitly warns that the two concepts are not identical. SIMD exposes a fixed-width vector instruction with one control-flow path. SIMT exposes individual threads to the programmer and groups them for execution.[1]

The programming model treats the warp as progressing together through the kernel. Hardware implementation details can be more flexible, especially on NVIDIA architectures with Independent Thread Scheduling, but portable code must not rely on undocumented sub-warp behavior.[1]

Warp grouping has several consequences:

- block sizes that are multiples of 32 avoid a permanently under-filled final warp;
- neighboring threads often issue memory operations together, which is why address layout affects coalescing;
- control-flow divergence is meaningful within a warp;
- many warp-level collective operations require explicit participation masks and synchronization rules.[1]

The warp is not “32 instructions running independently.” It is the scheduling/execution grouping through which 32 CUDA threads commonly make progress. Threads remain logical program entities; the warp is the hardware-facing execution group.

6 Divergence and active lanes

Suppose a warp executes a data-dependent branch:

```
if (x > 0) {
    path_A();
} else {
    path_B();
}
```

If all active threads choose the same branch, the warp follows one path. If some threads choose A and others choose B, the warp has diverged. CUDA's execution model must account for both paths while enabling only the lanes that belong to the path currently being executed.[1]

Divergence does not mean the kernel is incorrect, and it does not impose one fixed penalty. The cost depends on how much work the different paths contain, how they reconverge, and architecturespecific scheduling. The durable point is simpler: when only a subset of lanes performs useful work for an instruction, the warp's available lane-level throughput is not fully used.

Branch divergence occurs within a warp. Two different warps can take completely different paths without causing warp divergence between them.[1]

Independent Thread Scheduling, introduced on Volta-generation NVIDIA GPUs, makes thread scheduling more flexible than older lockstep implementations. It does not make arbitrary divergence free, nor does it remove the need to use the synchronization primitives required by CUDA's programming model.[1]

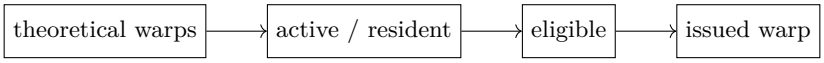
Divergence is also not the only way lanes become inactive. A final partial warp, predicated instructions, masks, or boundary conditions can leave lanes inactive without a conventional if/else split.

> Safe statement: divergence can reduce useful lane participation. Unsafe statement: "every divergent branch automatically makes a warp twice as slow." The actual cost depends on the executed paths and hardware behavior.

7 Warp schedulers and eligible warps

Residency alone does not guarantee progress. A resident warp may be waiting for a memory dependency, an arithmetic result, synchronization, or another event. A warp whose next instruction can issue is eligible. Schedulers select among eligible warps and issue instructions to the appropriate pipelines.[3]

Nsight Compute's scheduler model is useful because it separates four states:



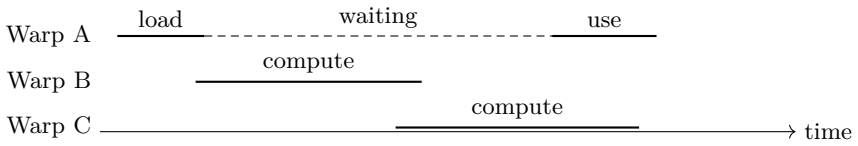
Theoretical residency is limited by launch configuration and resources. Active warps are resident. Eligible warps are active warps that are not stalled. Issued warps are those actually selected to send an instruction. Nsight notes that cycles with no eligible warp lead to skipped issue opportunities and indicate poor latency hiding.[3]

This distinction explains a common profiling surprise: a kernel can have respectable occupancy yet still issue inefficiently because too few active warps are eligible at the moments that matter. Conversely, a kernel can run well at moderate occupancy if it exposes enough instruction-level parallelism and keeps the relevant pipelines supplied.

The scheduler is therefore a bridge between architecture and performance. It cannot manufacture independence that the program does not expose. It can only choose among work that is resident and ready.

8 Latency hiding is not latency reduction

GPU operations have latency. A global-memory request, an arithmetic dependency, or synchronization can prevent one warp from issuing its next instruction immediately. GPUs cope with much of this latency by having other independent work available.[2,3]



Warp A's memory latency has not vanished. The machine simply did useful work from B and C during part of the waiting interval. This is latency hiding.[2]

Two kinds of independence help:

- thread-level parallelism (TLP): other warps have ready instructions;
- instruction-level parallelism (ILP): the same warp has independent operations that can proceed without waiting on the stalled result.

This is why occupancy matters but is not the whole story. More resident warps create more candidates for TLP. But if a kernel already has enough ready work to cover the important latencies, adding residency may not improve runtime.[2]

> A GPU often wins not because each memory access is low latency, but because the architecture is built to tolerate latency by maintaining and scheduling many independent operations.

9 Residency, execution, and progress

Three words that are often blurred together should stay separate.

Resident means a block or warp has been admitted to an SM and its required state is allocated. Registers, shared memory, block slots, warp slots, and thread limits constrain residency.[1,2]

Active is commonly used by profiling tools for resident warps that have not completed. An active warp may still spend cycles stalled.[3]

Eligible means a resident warp is ready to issue its next instruction rather than being blocked by a dependency or synchronization condition. Issued means the scheduler actually selected that warp to dispatch an instruction in that scheduling opportunity.[3]

$$\text{logical work} \supset \text{resident work} \supset \text{eligible work} \supset \text{issued work at a moment in time.}$$

A kernel can launch millions of logical threads while only a bounded set is resident. It can have many resident warps while only a few are eligible. It can have eligible warps while a particular execution pipeline is saturated and another is idle.

This separation is essential when interpreting performance counters. A residency metric cannot tell you which functional units were busy. A utilization metric based on “any kernel executing” cannot tell you whether the dominant pipeline was near peak. A throughput metric for one subsystem cannot tell you whether end-to-end application time was dominated by another subsystem.[3,4]

10 Occupancy: the exact definition

Occupancy is a residency metric. In CUDA it is computed from the number of warps active on an SM relative to that SM's architectural limit for active warps.[2]

$$\text{occupancy} = \frac{\text{active resident warps per SM}}{\text{maximum resident warps per SM}}.$$

If an architecture permits at most 64 resident warps on an SM and a kernel configuration allows 32 to be resident, the theoretical occupancy is 50%.

Occupancy is constrained jointly by several finite resources:

- threads per block and maximum resident threads;
- warps per block and maximum resident warps;
- maximum resident blocks;
- registers consumed by resident threads;
- shared memory consumed by resident blocks;
- architecture-specific resource allocation granularity.[1,2]

One limiting resource is enough to stop additional blocks from becoming resident. A kernel that barely uses shared memory can still be register-limited. A kernel with modest register use can be shared-memory-limited. Very small blocks can hit the block-count limit before filling the available thread slots.[1]

Occupancy is therefore a residency ratio, not the percentage of arithmetic units doing useful work. Nsight Compute even provides a dedicated Occupancy Calculator because the interaction of resource limits and allocation granularity can be non-obvious.[3]

> Occupancy answers "how much warp residency can this launch sustain?" It does not answer "how close is this kernel to peak compute throughput?"

11 Why 100% occupancy is not the goal

More resident warps can improve latency tolerance, but performance does not rise indefinitely with occupancy. Once a kernel already exposes enough ready work to cover its important stalls, additional residency may add little.[2]

Trying to force occupancy upward can also create a trade-off. Registers are useful because they keep thread-local state on chip. Reducing the allowed register count may admit more blocks, but it can also increase instruction count or spill values to local memory. A lower-occupancy kernel with enough ILP may outperform a higher-occupancy version that spills or does more work.[2]

$$\text{higher occupancy} \not\Rightarrow \text{proportionally higher performance.}$$

Low occupancy can still be diagnostic. If too few warps are resident, the scheduler has fewer alternatives when one warp stalls. But the right question is not "how do I maximize occupancy?" It is "is insufficient ready work preventing the bottlenecked resource from being used effectively?"

Profiling should connect occupancy to scheduler behavior and resource throughput. If issue slots are frequently skipped because there are no eligible warps, more latency-hiding capacity may help. If the relevant compute or memory path is already near its sustainable limit, raising occupancy may do nothing.[3]

> Treat occupancy as a constraint and diagnostic, not a score. The fastest kernel is the goal; maximum occupancy is only one possible means to that end.

12 Registers: fast, thread-local state

Registers are on-chip storage associated with an SM and logically private to each thread. The compiler uses them for thread-local values such as loop variables, addresses, accumulators, and intermediate arithmetic state.[1,2]

Registers matter for two opposite reasons. First, they are exactly where performance-sensitive values often should live: close to execution, without repeated traffic to larger memory. Second, the register file is finite and shared among all resident threads on an SM. High per-thread register use can therefore reduce how many blocks and warps fit concurrently.[1,2]

This is why register usage is a capacity problem as well as a latency problem. A kernel can improve local reuse by holding more values in registers but lose residency because fewer thread contexts fit. The optimal point is workload-specific.

Registers also participate in dependency chains. If an instruction consumes a value produced by a previous instruction, the consumer may need to wait until the result is available. Other independent instructions or warps can hide that latency. NVIDIA's Best Practices Guide specifically discusses hiding register dependencies through sufficient parallel work.[2]

For AI kernels, register tiling is central to high-performance matrix multiplication and fused kernels because partial results can be accumulated close to the arithmetic pipelines. But aggressive register use increases pressure. Later handbooks will treat the register/occupancy trade-off quantitatively; here the architectural lesson is that registers are both a high-value resource and a finite residency budget.

13 Register pressure, spilling, and local memory

When register demand is too high, or when certain thread-local objects cannot be kept in registers, the compiler can place them in CUDA local memory. Despite the name, this storage is per-thread only in visibility; it is backed by device memory rather than by a private on-chip memory.[2]

Register spilling is one common reason values reach local memory. CUDA may also place some large or dynamically indexed thread-local objects there. These accesses travel through the device-memory and cache path, so they are far more expensive than register access and can add substantial memory traffic.[2]

Reduce register count	**Possible consequence**
more blocks may become resident	occupancy may increase
less on-chip thread-local capacity	values may spill
more local-memory traffic	latency / bandwidth cost may rise

That is why `-maxrregcount` or launch bounds are tuning controls, not automatic improvements. NVIDIA explicitly warns that lower register limits can increase spilling even while increasing occupancy.[2]

The practical workflow is to inspect compiler resource reports and profile the resulting kernel, rather than assuming that fewer registers or more occupancy must be better.

14 Shared memory: software-managed on-chip storage

Shared memory is explicitly addressed on-chip storage that threads within a block can use for cooperation and data reuse. Software chooses what to place there and when to synchronize around it, unlike a hardware-managed cache.[1,2]

A common pattern is tiling: load a subset of a larger tensor from device memory into shared memory, synchronize if necessary, reuse that tile for multiple operations, then move on. The goal is not simply to use a "faster memory." The goal is to reduce expensive repeated movement from farther away in the hierarchy.

Shared memory also consumes an SM residency budget. A block that requests a large allocation may prevent other blocks from being resident simultaneously. Thus shared memory can improve locality while reducing occupancy. Both effects are real.[1,2]

On recent NVIDIA architectures, shared memory and L1 functionality can share or partition a unified on-chip data-cache resource, but the exact size and organization vary by compute capability. That implementation fact does not turn shared memory into an ordinary cache: the programmer still allocates and addresses shared memory explicitly.[1]

Synchronization is another part of the cost model. If threads cooperate through shared memory, barriers such as `__syncthreads()` may be required for correctness. The data reuse benefit must therefore be weighed against storage pressure, bank-access behavior, and synchronization overhead.

15 Caches: L1 and L2 are not shared memory

GPUs also contain hardware-managed caches. In CUDA's device memory model, global memory traffic can pass through cache structures, including per-SM L1 functionality and a device-wide L2 cache. Exact policies and capacities depend on architecture.[1]

The key distinction is control:

	Shared memory	**Cache**
management	explicit in program	hardware-managed
scope	block-local; cluster DSM if supported	transparent hierarchy
use	deliberate staging / reuse	temporal / spatial locality
residency	allocation can limit blocks	no per-block shared-memory budget

Caches can reduce the traffic that reaches HBM/device DRAM, but they are not guaranteed to hold every useful working set. Access pattern, reuse distance, contention, and architecture all matter.

This is why "the tensor fits in GPU memory" says almost nothing about kernel performance. Capacity tells us whether data can reside somewhere in the memory system. Performance depends on how often data moves, how much reuse is captured on chip, and whether the working set and access pattern match the hardware.

Later handbooks will treat the hierarchy quantitatively. For AS01, remember the roles: registers are thread-local state, shared memory is explicit on-chip cooperation/reuse, caches are hardwaremanaged locality, and device memory provides large capacity at much greater distance from the execution pipelines.

16 HBM and device memory: capacity is not bandwidth

CUDA's programming model uses global memory for the device-attached DRAM visible to all SMs. Many data-center AI accelerators implement this large memory with HBM technology, so AI systems discussions often use "HBM" as shorthand for the far, high-capacity level of GPU memory.[5,10]

Two quantities must be separated:

- capacity: how many bytes can be stored;
- bandwidth: how many bytes per second can be transferred under a given access pattern and workload.

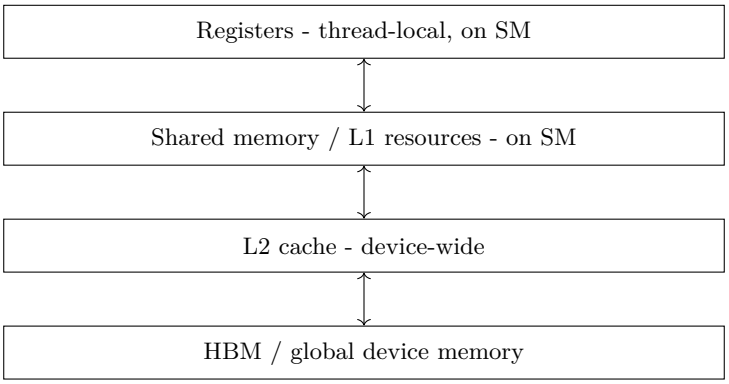
A model can fit entirely in device memory and still run slowly because kernels repeatedly move large amounts of data through the HBM interface. Conversely, a kernel may have a small memory footprint yet be latency-sensitive because its accesses are sparse or dependent.

Peak memory bandwidth is also a ceiling, not a promise. Effective bandwidth depends on access coalescing, cache behavior, instruction mix, outstanding requests, and other details. CUDA's Best Practices Guide recommends measuring effective bandwidth rather than assuming the theoretical specification is achieved.[2]

> "Fits in HBM" is a capacity statement. "HBM is saturated" is a bandwidth statement. "The kernel is waiting on memory" is a latency/dependency statement. They are related but not interchangeable.

17 The memory hierarchy is a data-movement problem

A useful AI-oriented memory hierarchy is:



The hierarchy is not merely a ranking of "fast" and "slow" locations. Each level has different capacity, scope, management policy, bandwidth, and access semantics. The performance question is how many bytes move between levels and how much useful computation is extracted from data while it is nearby.

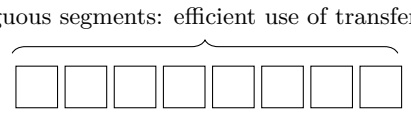
FlashAttention is a useful example: it computes exact dense attention but changes the execution schedule. Intermediate tiles are consumed and reused near the compute units instead of repeatedly round-tripping large intermediates through HBM.[12]

The same idea appears in GEMM tiling, fused elementwise kernels, convolutions, reductions, and many compiler optimizations. Reuse matters because moving a value once and using it many times can be cheaper than fetching the same value repeatedly from farther away.

This also explains why FLOP count alone is a poor runtime predictor. Two implementations can perform similar arithmetic but generate different memory traffic. A workload's data movement is part of its execution algorithm, even when the mathematical expression on paper does not mention memory at all.

18 Coalescing: how a warp asks for memory

When threads in a warp issue global-memory accesses, the GPU combines their requests into memory transactions. CUDA calls this coalescing. Efficient access uses the bytes brought in by each transaction rather than fetching many segments that contain little requested data.[1,2]



A scattered pattern can require more transactions and transfer bytes that the warp never uses. This wastes bandwidth even though the program requested the same number of useful values.[2]

Coalescing therefore connects thread mapping to memory performance. The same tensor can be cheap or expensive to read depending on which dimension neighboring lanes traverse and how the data is laid out. AS07 will treat this deeply; AS01 only needs the architectural insight: memory traffic is generated collectively by groups of threads, so address pattern matters.

19 Execution pipelines: instructions do not use one resource

An SM contains multiple execution paths. A floating-point add, integer operation, load/store instruction, special function, and matrix multiply-accumulate are not all serviced by one undifferentiated "core." Exact pipeline counts and mappings change across GPU generations.[1,5,6]

This matters because utilization can be asymmetric. A kernel might saturate one subsystem while others are mostly idle:

- a dense GEMM may spend much of its useful time feeding matrix-multiply hardware;
- a memory copy can approach memory-system limits while doing very little arithmetic;
- an elementwise kernel can be limited by memory traffic or instruction issue rather than Tensor Cores;
- a reduction can be constrained by dependencies and synchronization.

Therefore, "the GPU has unused compute" is not a complete diagnosis. Which compute? Which instruction path? Is memory already the bottleneck? Are there eligible warps? Is the kernel too small to expose enough device-wide parallelism?

Nsight Compute's throughput model reflects this reality by reporting high-level compute and memory throughput relative to peak and exposing lower-level contributors.[3]

> A GPU is a collection of coupled subsystems. Performance tuning means finding the subsystem that limits useful progress for a specific workload, not trying to make every block in a diagram read 100%.

20 Tensor Cores: matrix MMA, not magic AI cores

Tensor Cores are specialized hardware for matrix multiply-accumulate (MMA) operations. NVIDIA's programming interfaces expose operations conceptually of the form

$$D = A \times B + C.$$

The supported tile shapes, operand formats, accumulation formats, and execution scope depend on GPU generation.[7,11]

Tensor Cores matter enormously for AI because training and inference contain large GEMMs. But they do not accelerate every instruction in a model. Indexing, control flow, many reductions, elementwise transformations, memory movement, synchronization, and host-side work use other resources.

The word "Tensor Core" also hides generational change. Earlier CUDA interfaces exposed warp-level matrix operations. Hopper introduced asynchronous warpgroup MMA instructions for certain high-performance paths, where four contiguous warps cooperate as a warpgroup. Datacenter Blackwell SM100 adds newer tcgen05.mma instructions and Tensor Memory (TMEM) for parts of the matrix pipeline.[6,11]

These differences are implementation details around a durable principle: specialized matrix hardware achieves high throughput when the workload is expressed in supported matrix tiles and data formats and when the surrounding data movement keeps the units fed.

> Tensor Cores raise the ceiling for supported matrix arithmetic. They do not remove memory traffic, launch overhead, synchronization, or non-matrix work from an AI workload.

21 Precision changes the available execution path

Peak arithmetic throughput depends on data type. A GPU can have very different theoretical rates for FP64, FP32, TF32, BF16, FP16, FP8, integer formats, and newer narrow formats. Matrix hardware also supports specific combinations of input and accumulation precision that vary by generation.[5,6,7,10]

This means a specification such as “X TFLOPs” is incomplete unless it identifies at least:

- the operation class;
- input data type;
- accumulation mode;
- dense vs structured-sparse assumptions where applicable;
- hardware generation.

Precision is not only a performance knob. It changes numerical behavior, memory footprint, memory traffic, and sometimes which execution instructions are legal. An FP16/BF16 training kernel may use matrix paths unavailable to an FP64 scientific kernel. An FP8 path may require scaling and error-management techniques that a higher-precision path does not.

For systems reasoning, keep two claims separate: the hardware supports a high-throughput lowprecision path and this model or kernel can safely and efficiently use it. The first is architectural. The second depends on software and numerical requirements.

22 How GEMM maps onto the GPU hierarchy

Matrix multiplication is a useful architectural example because high-performance implementations deliberately mirror the GPU hierarchy. CUTLASS describes GEMM as a hierarchy of tiles: devicewide work is decomposed into thread-block tiles, then warp/warpgroup tiles, then thread or matrixinstruction fragments.[11]

A simplified dataflow is:



The point of tiling is reuse. A tile loaded from global memory can be reused across many multiplyaccumulate operations before being evicted. Registers hold accumulators or fragments close to the math units. Shared memory provides a cooperative staging area.[11]

Modern generations add more asynchronous machinery to overlap data movement with matrix computation. Hopper's WGMMA/TMA ecosystem and datacenter Blackwell SM100's tcgen05/TMEM paths are examples.[5,6,11]

The mathematical GEMM C = AB does not describe any of this. The implementation schedule determines where tiles live, when they move, how many warps cooperate, and whether the expensive matrix units are continuously supplied. That gap between equation and execution is one of the central themes of AI systems engineering.

23 Instruction-level and thread-level parallelism

A GPU can hide latency using parallelism at more than one scale.

Thread-level parallelism (TLP) provides many warps. If Warp A stalls, the scheduler may issue from Warp B.

Instruction-level parallelism (ILP) provides independent operations within one thread/warp. If one dependency chain is waiting, another independent instruction may be available.

These mechanisms interact. High occupancy increases the pool of warps that can contribute TLP. Rich ILP can reduce the number of resident warps required to keep pipelines busy. This is one reason low occupancy is not automatically slow. NVIDIA's Best Practices Guide explicitly notes that sufficient ILP can sometimes cover latency even at lower occupancy.[2]

Dependencies matter because an instruction cannot consume a result before that result is ready. Memory dependence is only one source; arithmetic pipelines also have latency. Scheduling attempts to fill the gaps with independent work.[2,3]

The practical consequence is that performance cannot be predicted from thread count alone. Ten thousand logical threads may still expose poor ready-work behavior if they all hit the same dependency pattern. Conversely, a carefully pipelined kernel can keep specialized resources busy with a smaller resident set.

24 Synchronization can expose idle time

Threads cooperate, so some kernels require synchronization. CUDA's `__syncthreads()` is a block-wide barrier: every non-exited thread in the block must reach the same barrier instance, or exit, before waiting threads can continue. It also provides the required memory ordering among participating threads.[1]

Barriers are essential for correctness in patterns such as “load a shared-memory tile, then let every thread consume it.” But a barrier can expose imbalance. If some warps reach the barrier early while others are still doing work, the early warps wait.

Synchronization cost is therefore not just the instruction itself. It includes the amount of useful work that becomes unavailable while threads wait for the slowest required participants.

Modern CUDA also provides warp-level synchronization, cooperative groups, asynchronous barriers, and architecture-specific mechanisms that allow more sophisticated producer/consumer pipelines.[1]

High-performance AI kernels use these tools to overlap stages, but the fundamental rule remains: synchronization creates dependencies between workers. The schedule must respect those dependencies, and unnecessary synchronization can reduce available parallelism.

> A kernel with many resident warps can still have poor latency hiding if most of those warps are waiting at the same synchronization point. Residency is useful only when it creates ready work.

25 Kernel launches and asynchronous execution

CUDA kernel launches are asynchronous with respect to the host thread: after enqueueing a kernel, the CPU can continue while device work is still pending. Code that needs the result, or wants to measure completed kernel time, must synchronize at the appropriate point.[1]

This affects both performance and measurement. A naive CPU timer placed around a launch can measure mostly launch overhead rather than kernel runtime unless synchronization is handled correctly. CUDA's Best Practices Guide therefore distinguishes CPU timing from GPU event timing and emphasizes the asynchronous behavior of the API.[2]

Launch overhead becomes especially relevant when work is fragmented into many tiny kernels. The GPU may have enormous arithmetic capacity, but if each launch performs too little work, host dispatch and kernel boundaries can become visible in wall-clock time. CUDA Graphs and fusion are later-series topics that address parts of this problem.

Independent streams can overlap when resources and dependencies permit. Newer GPU mechanisms can also overlap data movement with computation inside kernels.[1,5,11]

Host and device execution are decoupled; synchronize only where correctness or measurement requires it.

26 What “GPU utilization” actually means

In NVML, the commonly surfaced device gpu utilization value is an activity-time statistic: over each sample window, it records the share of time for which the GPU had at least one kernel executing.[4]

That metric does not mean:

- 100% of arithmetic units were busy;
- Tensor Cores were saturated;
- HBM bandwidth was saturated;
- every SM was fully occupied;
- the workload was close to optimal throughput.

A kernel can execute continuously and still leave large parts of the GPU underused. For example, it might use only a small subset of SMs, issue inefficiently, or be limited by one pipeline while other resources sit idle.

NVML’s memory field is likewise time-based: it records the share of the sample window in which global device-memory reads or writes were occurring. It should not be interpreted as achieved bandwidth divided by peak bandwidth.[4]

For optimization, kernel profilers such as Nsight Compute expose more specific resource-throughput, occupancy, scheduler, cache, and memory metrics.[3]

A dashboard can correctly show 100% “GPU utilization” while a kernel is still far from the performance ceiling that matters for its workload.

27 Occupancy, activity, and throughput are different

Several percentages can appear in GPU tooling, and they answer different questions.

Metric family	What it is trying to describe
occupancy	resident warp fraction relative to the architectural limit
NVML GPU utilization	share of the sample window with kernel execution
active SM / pipeline activity	whether a unit was active over measured cycles
eligible / issued warps	scheduler readiness and actual issue opportunities, separate from residency
throughput vs peak	achieved activity relative to an architecture-specific ceiling

Nsight Compute's Speed Of Light analysis reports compute and memory throughput as percentages of appropriate peak rates. Its scheduler sections separately report active, eligible, and issued warp behavior.[3]

These numbers can disagree without contradiction. A kernel might have 100% NVML utilization because it runs continuously, 50% occupancy because register use limits residency, high memory throughput because HBM is near its ceiling, and modest arithmetic throughput because the kernel is bandwidth-bound.

That combination could describe an efficient kernel. Trying to raise arithmetic utilization would not help if memory is already the limiting resource.

> Metrics are only meaningful after you ask what quantity they measure, over what time interval, and relative to which denominator.

28 Peak TFLOPs vs achieved throughput

Peak TFLOPs is a hardware ceiling for a particular operation and precision under specified assumptions. It is not application throughput.[2,5,6]

To approach peak matrix throughput, a workload must provide supported data types and tile shapes, enough parallel work, high operand reuse, sufficient instruction issue, and a data pipeline that keeps the matrix units supplied. Real models also execute non-matrix kernels and incur memory movement, synchronization, communication, launch, and framework overhead.

For the same operation, precision, and counting convention, hardware peak is a ceiling on sustained arithmetic throughput. End-to-end application performance additionally reflects memory movement, synchronization, communication, kernel launches, and non-matrix work.

Benchmark scope matters. A microbenchmark that measures one GEMM kernel is not directly comparable to tokens/s for a serving system. An achieved TFLOP/s number for one dtype is not comparable to another dtype's peak. A vendor's sparse peak is not the same as dense peak.

Nsight Compute's throughput metrics are useful because they explicitly normalize measured subsystem activity against architecture-specific peak rates, while still requiring the engineer to identify which resource is actually limiting the kernel.[3]

> Never read a peak number as "the GPU will deliver this for my model." Peak is a ceiling for a specific hardware path; application throughput includes everything required to produce the result.

29 Compute-bound and memory-bound are workload properties

A kernel needs both arithmetic and data movement. One useful first-order quantity is arithmetic intensity for a chosen memory level:

$$I = \frac{\text{operations}}{\text{bytes moved across that memory level}}.$$

The original Roofline formulation uses off-chip memory traffic (operational intensity) to relate attainable performance to peak compute and memory bandwidth. Hierarchical variants can apply the same style of reasoning to additional levels.[13]

High-intensity kernels reuse data enough that arithmetic throughput can become the limiting ceiling. Low-intensity kernels move many bytes for relatively little computation and are more likely to be constrained by memory bandwidth. The exact boundary depends on the hardware.[13]

Two caveats matter in AS01.

1. "Memory-bound" does not mean "uses lots of memory capacity." It means progress is limited by a memory-system resource under the measured workload.

2. A kernel is not permanently compute-bound or memory-bound by name. Shape, dtype, batch size, cache behavior, fusion, and GPU generation can change the bottleneck.

The full Roofline model belongs in AS04. For architecture, the lesson is enough: a GPU's compute and memory systems provide separate ceilings. Improving the non-limiting ceiling does not necessarily make a workload faster.

30 AI workloads stress different parts of the GPU

“AI workload” is not one performance regime.

Large dense GEMMs can achieve high arithmetic intensity through tiling and reuse, making matrix throughput and the surrounding data pipeline central.

Elementwise kernels may perform only a few operations per element and therefore spend much of their cost moving data. Fusion can help by avoiding round-trips of intermediate tensors.

Reductions and normalization introduce dependencies and collective behavior that differ from embarrassingly parallel elementwise work.

Embedding or indexing workloads can have irregular access patterns that reduce locality or coalescing.

Attention contains matrix multiplications, softmax/reduction behavior, and potentially large intermediate traffic. FlashAttention demonstrates that changing the IO schedule without changing dense attention’s mathematical result can materially improve performance.[12]

Autoregressive serving changes shape from long-prompt prefill to one/few-query decode steps, so the same model can stress hardware differently across phases. The exact bottleneck depends on batching, context length, kernel choice, and system design.

This variety is why a model-level “GPU utilization” number is too coarse for serious diagnosis. A serving trace may alternate between kernels with entirely different bottlenecks.

31 NVIDIA vs AMD: names differ, principles persist

CUDA terminology is not the definition of a GPU. AMD exposes similar broad concepts with different names and some different widths and mechanisms. HIP explicitly warns portable code not to assume one fixed warp size. AMD calls the hardware grouping a wavefront; its size varies by architecture, while NVIDIA CUDA's warp size is 32.[8]

Concept	**NVIDIA language**	**AMD language**
execution engine	Streaming Multiprocessor (SM)	Compute Unit (CU)
thread group	warp	wavefront / warp in HIP abstraction
programmer-managed on-chip storage	shared memory	LDS (Local Data Share)
matrix acceleration	Tensor Core / MMA families	Matrix Core / MFMA families
programming stack	CUDA	ROCm / HIP

AMD's current performance guidance describes the same high-level occupancy trade-off: registers and shared/LDS resources constrain concurrent blocks/waves, and higher occupancy can help hide latency.[9]

The architectural lesson is therefore portable: expose parallel work, schedule it in hardware groups, provision per-thread and per-block state, move data through a hierarchy, and feed specialized execution units. The names, exact widths, resource limits, and instructions are vendorand generation-specific.

32 Architecture generations change the mechanisms

A handbook on GPU architecture can become obsolete if it confuses one generation's implementation with timeless theory. The durable principles are warps/wavefronts, residency, scheduling, data movement, execution pipelines, and resource constraints. The mechanisms evolve.

Hopper introduced TMA, thread-block clusters, and warpgroup-level asynchronous MMA for high-performance kernels.[5,11]

Datacenter Blackwell SM100 introduced tcgen05.mma Tensor Core instructions and a distinct Tensor Memory (TMEM) used by those matrix pipelines. CUTLASS treats these as architecture-conditional capabilities rather than universal Blackwell or CUDA behavior, and explicitly distinguishes datacenter SM100 from GeForce Blackwell SM120.[6,11]

AMD CDNA generations likewise evolve matrix formats, compute-unit structure, packaging, and memory systems. CDNA 3, for example, extends Matrix Core datatype support and underlies the MI300 accelerator family.[10]

This evolution is why architecture-specific details in this handbook are labeled and why later performance work must be tied to the actual deployed GPU and software version.

> Do not memorize one block diagram as "the GPU." Memorize the invariants, then consult the architecture documentation for the generation-specific realization.

33 Worked execution example: 61,440 logical threads

Consider a CUDA kernel launched with

240 blocks	256 threads per block

The grid contains

$$240 \times 256 = 61{,}440$$

logical threads. Because an NVIDIA warp contains 32 threads, each block contains

$$256/32 = 8$$

warps, and the complete grid contains

$$240 \times 8 = 1{,}920$$

warps of logical work. Those 1,920 warps are not all resident at once. Blocks are dispatched across SMs as resources become available. How many blocks fit on one SM depends on the device's architectural limits and this kernel's resource requirements, especially registers and shared memory.[1,2]

Once blocks are resident, warp schedulers choose eligible warps and issue their instructions to the appropriate execution paths. Some warps may wait on memory or dependencies while others proceed. As blocks complete, their resources are freed and later blocks can become resident.

This example separates three scales:

- 61,440 threads: logical parallel work;
- 1,920 warps: NVIDIA execution groupings across the whole grid;
- resident warps at one time: bounded by the actual device and kernel resource use.

The arithmetic is simple, but the distinction is fundamental. Launch size describes work to be done, not instantaneous physical parallelism.

34 Worked occupancy example: resource limits interact

Use a deliberately hypothetical SM so the arithmetic illustrates the mechanism without pretending to describe a specific GPU. Suppose it allows at most 64 resident warps, has enough block slots for the configurations below, and no additional allocation-granularity limit. It contains 65,536 32-bit registers and 128 KiB of shared memory.

A kernel uses 256 threads per block, 64 registers per thread, and 32 KiB shared memory per block.

$$256/32 = 8 \text{ warps per block}$$

Register demand per block is

$$256 \times 64 = 16{,}384 \text{ registers},$$

so registers permit at most

$$65{,}536/16{,}384 = 4 \text{ blocks}.$$

Shared memory also permits

$$128 \text{ KiB}/32 \text{ KiB} = 4 \text{ blocks}.$$

Therefore four blocks are resident in this simplified model, or $4 \times 8 = 32$ warps, and occupancy is $32/64 = 50\%$. If shared memory were reduced to 16 KiB, occupancy would not automatically rise: registers would still cap residency at four blocks. Optimizing the non-limiting resource changes nothing. This is exactly why occupancy analysis must identify the active constraint rather than tune one number in isolation.[2,3]

35 Common misconceptions

“A GPU is just thousands of tiny CPUs.”

No. The threading, scheduling, memory, and throughput design is fundamentally different.[1]

“More CUDA cores means proportionally faster AI.”

No. AI kernels use different pipelines, memory resources, and precisions; core count alone is not a performance model.

"100% GPU utilization means the GPU is saturated." No. NVML’s commonly displayed value is an activity-over-time measure, not a saturation score.[4]

“Tensor Cores accelerate the whole model.”

No. They accelerate supported matrix MMA operations. Other work uses other resources.[7,11]

“If the model fits in HBM, memory is solved.”

No. Capacity, traffic, bandwidth, and latency are different constraints.

“Shared memory is just a cache.”

No. It is explicitly managed by the program, even when hardware shares physical resources with L1.[1]

“Latency hiding removes latency.”

No. It overlaps other work with the waiting interval.[2]

“AI kernels are compute-bound.”

Some are. Others are bandwidth-, latency-, issue-, synchronization-, or launch-limited depending on shape and implementation.

36 A practical mental model

A compact way to reason about a GPU AI workload is:

1. Expose work. A kernel defines a grid of blocks and threads.

2. Place blocks. Hardware admits blocks onto SMs subject to register, shared-memory, thread, warp, and block limits.

3. Form execution groups. NVIDIA blocks become 32-thread warps; other vendors use their own grouping rules.

4. Keep state close. Registers and shared memory hold useful working state while caches reduce traffic farther down the hierarchy.

5. Schedule readiness. Warp schedulers choose eligible work so one stalled dependency need not stall the entire SM.

6. Feed the right pipelines. Scalar/vector arithmetic, load/store paths, matrix units, and other resources handle different instructions.

7. Move data deliberately. Coalescing, tiling, caching, reuse, and fusion determine how much traffic reaches device memory.

8. Measure the bottleneck. Occupancy, utilization, bandwidth, issue efficiency, and achieved compute are different metrics. Use the one that answers the current question.

The engineering payoff is a better question. Instead of asking "why isn't my GPU at 100%?", ask:

Which resource or dependency limits useful progress for this workload, and what evidence shows that?

That question scales from a single CUDA kernel to attention, model training, distributed execution, and LLM serving. It is the foundation for the rest of Understanding AI Systems.

References and further reading

[1] NVIDIA. *CUDA Programming Guide*, current documentation, accessed September 2026. https://docs.nvidia.com/cuda/cuda-programming-guide/

[2] NVIDIA. *CUDA C++ Best Practices Guide*, Release 13.4 documentation, accessed September 2026. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/

[3] NVIDIA. *Nsight Compute: Profiling Guide and User Guide*, current documentation. https://docs.nvidia.com/nsight-compute/

[4] NVIDIA. *NVML API Reference: nvmlUtilization_t.* https://docs.nvidia.com/deploy/nvml-api/api/structnvmlUtilization__t.html

[5] NVIDIA. *Hopper Tuning Guide.* https://docs.nvidia.com/cuda/hopper-tuning-guide/

[6] NVIDIA. *CUTLASS: Blackwell SM100 GEMMs / tcgen05 functionality.* https://docs.nvidia.com/cutlass/latest/media/docs/cpp/blackwell_functionality.html

[7] NVIDIA. CUDA Programming Guide matrix / WMMA APIs; see also the Turing Tuning Guide for the historical Tensor Core programming model. https://docs.nvidia.com/cuda/cuda-programming-guide/

[8] AMD. *HIP C++ Language Extensions*, warpSize and cross-lane behavior, current ROCm documentation. https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/hip_cpp_language_extensions.html

[9] AMD. *HIP Performance Guidelines*, occupancy and resource guidance, current ROCm documentation. https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/performance_guidelines.html

[10] AMD. *Introducing AMD CDNA 3 Architecture*, white paper. https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf

[11] NVIDIA. *CUTLASS Documentation*: Efficient GEMM, Warpgroup MMA, and tcgen05 programming guides. https://docs.nvidia.com/cutlass/latest/

[12] Dao, T.; Fu, D. Y.; Ermon, S.; Rudra, A.; Ré, C. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 2022. https://arxiv.org/abs/2205.14135

[13] Williams, S.; Waterman, A.; Patterson, D. *Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures.* UCB/EECS-2008-134; CACM 2009. https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

Reference map: what to read for which question

Question	Best starting source
What is the current CUDA thread/block/warp execution model?	CUDA Programming Guide [1]
How are occupancy, registers, shared memory, and latency hiding related?	CUDA Best Practices Guide [2]
What do active, eligible, and issued warps mean in profiling?	Nsight Compute [3]
What exactly does the common NVIDIA "GPU utilization" percentage measure?	NVML nvmlUtilization_t [4]
Which mechanisms are Hopper-specific?	Hopper Tuning Guide [5] and CUTLASS [11]
Which mechanisms distinguish datacenter Blackwell SM100-class paths from SM120?	CUTLASS Blackwell / tcgen05 docs [6,11]
What does a Tensor Core matrix operation actually compute?	Turing/CUDA matrix documentation [7]
Why should portable GPU code not assume one warp width across vendors?	AMD HIP language extensions [8]
How does AMD describe occupancy and latency hiding?	AMD HIP performance guidance [9]
What is an AMD AI-accelerator architecture example?	AMD CDNA 3 white paper [10]
How does a high-performance GEMM map onto blocks, warps, shared memory, and registers?	CUTLASS GEMM documentation [11]
Why can less memory traffic make the same attention mathematics faster?	FlashAttention [12]
How do compute ceilings and memory bandwidth interact?	Roofline model [13]

Implementation support and hardware details change faster than the architectural principles. For deployed systems, treat the documentation for the exact GPU generation, toolkit, compiler, framework, and profiler version as authoritative.